

Doc. no.: P0602R0

Date: 2017-02-04

Audience: Library Evolution Working Group

Reply-to: Zhihao Yuan <zy at miator dot net>

variant and optional should propagate copy/move triviality

Rationale

Making variant and optional conditionally trivially copy constructible, trivially move constructible, trivially copy assignable, and trivially move assignable significantly improves codegen and performance when they are nested or being put into containers.

MS STL variant makes those four operations trivial if present while all alternatives are trivially copyable. libc++ makes them individually trivial if all alternatives have the corresponding operations being trivial. Balancing the implementation complexity and the usefulness in practice, MS STL's approach is proposed here. This approach is forward-compatible with libc++'s full-fledged approach.

Wording

This wording is relative to N4618.

Modify 20.6.3.1 [optional.ctor] as follows:

```
optional(const optional& rhs);
```

[...]

Remarks: This constructor shall be defined as deleted unless `is_copy_constructible_v<T>` is true. If not deleted, this constructor shall be trivial if `is_trivially_copyable_v<T>` is true.

```
optional(optional&& rhs) noexcept(see below );
```

[...]

Remarks: The expression inside `noexcept` is equivalent to `is_nothrow_move_constructible_v<T>`. This constructor shall not participate in overload resolution unless `is_move_constructible_v<T>` is true. If it does, this constructor shall be trivial if `is_trivially_copyable_v<T>` is true.

Modify 20.6.3.3 [optional.assign] as follows:

```
optional<T>& operator=(const optional& rhs);
```

[...]

Remarks: If any exception is thrown, [...]. This operator shall be defined as deleted unless `is_copy_constructible_v<T>` is true and `is_copy_assignable_v<T>` is true. If not deleted, this assignment operator shall be trivial if `is_trivially_copyable_v<T>` is true.

```
optional<T>& operator=(optional&& rhs) noexcept(see below );
```

[...]

Remarks: The expression inside `noexcept` is equivalent to: [...]. This operator shall not participate in overload resolution unless `is_move_constructible_v<T>` is true and `is_move_assignable_v<T>` is true. If it does, this assignment operator shall be trivial if `is_trivially_copyable_v<T>` is true.

Modify 20.7.2.1 [variant.ctor] as follows:

```
variant(const variant& w);
```

[...]

Remarks: This ~~function shall not participate in overload resolution~~ constructor shall be defined as deleted unless `is_copy_constructible_v<Ti>` is true for all *i*. If not deleted, this constructor shall be trivial if `is_trivially_copyable_v<Ti>` is true for all *i*.

```
variant(variant&& w) noexcept(see below );
```

[...]

Remarks: The expression inside `noexcept` is equivalent to [...]. This function shall not participate in overload resolution unless `is_move_constructible_v<Ti>` is true for all *i*. If it does, this constructor shall be trivial if `is_trivially_copyable_v<Ti>` is true for all *i*.

Modify 20.7.2.3 [variant.assign] as follows:

```
variant& operator=(const variant& rhs);
```

[...]

Remarks: This ~~function shall not participate in overload resolution~~ operator shall be defined as deleted unless `is_copy_constructible_v<Ti> && is_move_constructible_v<Ti> && is_copy_assignable_v<Ti>` is true for all *i*. If not deleted, this assignment operator shall be trivial if `is_trivially_copyable_v<Ti>` is true for all *i*. [...]

```
variant& operator=(variant&& rhs) noexcept(see below );
```

[...]

Remarks: This function shall not participate in overload resolution unless `is_move_constructible_v<Ti> && is_move_assignable_v<Ti>` is true for all *i*. If it does, this assignment operator shall be trivial if `is_trivially_copyable_v<Ti>` is true for all *i*. The expression inside `noexcept` is equivalent to: [...]